Now, it's time to put these new skills to the test. *Don't skip this one;* you're going to build on this work in the next chapter.

## Exercises

In these exercises, you're going to implement another key piece of your application at the routing layer. Along the way, you'll be looking for opportunities to share code.

### Reducing Duplication

Search through your unit specs for JSON.parse; you will see several snippets that look like this:

05-unit-specs/11/expense_tracker/spec/unit/api_example_spec.rb

```
parsed = JSON.parse(last_response.body)
expect(parsed).to do_something
```

We're repeatedly grabbing the last_response from Rack::Test, and then parsing it from JSON. Find a way to reduce this duplicated logic.

### Implementing the GET Route

Through this chapter, we built a spec for the POST route together. Now, it's time to build one for the GET route and make it pass. This will be much easier than the POST version; all the pieces are already in place.

The goal is to be able to hit a URL containing a date:

```
get '/expenses/2017-06-12'
```

…and get back JSON data containing whatever expenses were recorded on that day.

First, add the following outline to spec/unit/app/api_spec.rb, *inside* the RSpec.describe API block:

05-unit-specs/11/expense_tracker/spec/unit/app/api_spec.rb

```
describe 'GET /expenses/:date' do
  context 'when expenses exist on the given date' do
    it 'returns the expense records as JSON'
    it 'responds with a 200 (OK)'
  end

  context 'when there are no expenses on the given date' do
    it 'returns an empty array as JSON'
    it 'responds with a 200 (OK)'
  end
end
```